Generating 2D Datasets With Similar Statistical Properties

Algorithm Engineering 2025 Project Paper

Justin Bergmann
Friedrich-Schiller-Universität Jena
Germany
justin.bergmann@uni-jena.de

Abstract

Summary statistics such as mean, standard deviation, and correlation can be highly misleading, as demonstrated by the "Datasaurus Dozen" dataset. This paper presents a high-performance C++23 implementation of a Simulated Annealing algorithm designed to transform 2D point clouds into complex target geometries while preserving their original descriptive statistics. By introducing an incremental statistical update logic with $O(1)$ complexity and leveraging OpenMP for parallelization, the implementation achieves a throughput of 2.4×10^7 to 3.6×10^7 iterations per second, depending on the target shape. Experiments show that even complex high-density fractal structures like the Sierpinski triangle ($N = 1420$) can be generated with extreme precision over 10^9 iterations, ensuring the statistics remain stable up to the third decimal place.

Keywords

Datasaurus Dozen, Simulated Annealing, Algorithm Engineering, Statistical Preservation, C++23

1 Introduction

> "I only believe in statistics that I have doctored myself."
>
> Commonly attributed to Winston Churchill

Is one of the most famous quotes about statistics and highlights the importance of understanding the underlying data rather than relying solely on summary statistics, as famously demonstrated by Anscombe's Quartet [1]. A famous example of this is the "Datasaurus Dozen" introduced by Matejka and Fitzmaurice [2], which demonstrates that datasets with nearly identical descriptive statistics (mean, standard deviation, and Pearson correlation) can exhibit drastically different visual structures. Generating such datasets is not only a matter of academic interest but also a challenge in algorithm engineering, as it requires balancing strict statistical constraints with iterative geometric transformations.

1.1 Background

The core problem in generating datasets with similar statistical properties lies in moving points within a 2D space toward a target shape without altering the global summary statistics beyond a certain threshold. Typically, this is achieved through a stochastic optimization process. However, as the number of points and the complexity of the target shapes increase, the computational cost of checking the statistics and calculating geometric distances grows significantly. To make this process viable for high-resolution datasets or complex animations, a fast and reliable software tool is required.

1.2 Related Work

The foundation of this work is the research by Matejka and Fitzmaurice [2], who introduced the use of Simulated Annealing to generate datasets with varied appearances and identical statistics. Their approach demonstrated that by repeatedly moving points and accepting changes based on a Metropolis criterion, one can "morph" a source dataset (like the Datasaurus) into various target shapes. While their original implementation proved the concept, modern hardware and algorithmic optimizations offer significant potential for performance enhancements. Alternative metaheuristics such as genetic algorithms could also be applied here. However, Simulated Annealing is a more natural fit because it works with single-point movements, which makes it straightforward to check and enforce the statistical constraints after each move.

1.3 My Contributions

This paper presents a high-performance C++23 implementation of the Simulated Annealing approach for statistical preservation. My main contributions include:

- $O(1)$ **Statistical Updates:** Instead of recomputing the dataset's statistics in $O(n)$ time after each perturbation, I implement an incremental update logic that maintains summary statistics in constant time.
- **Spatial Indexing:** I utilize a grid-based lookup table (Spatial Grid) to accelerate the distance calculations between points and target shapes, making the runtime independent of the target shape's point density.
- **Parallelization:** I use OpenMP to run multiple independent optimization chains in parallel, utilizing all available CPU cores to explore a larger portion of the solution space and improve result quality.

1.4 Outline

I describe the implemented Simulated Annealing algorithm in §2. The implementation details, including the $O(1)$ update logic, spatial indexing, parallelization strategy, and vectorization considerations, are presented in §3. I present the results of my experiments regarding runtime and quality in §4, before concluding my work in §5.

2 The Algorithm

The core optimization is driven by the Simulated Annealing algorithm, a randomized optimization method by Kirkpatrick et al. [3]. This method is particularly effective at finding the global optimum

AE 2026, March 1, Jena, Germany. Copyright ©2026 for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

of a given function while avoiding local minima. In this section, I describe how the algorithm transforms a set of data points into a new shape while keeping the statistical values constant.

2.1 The Core Concept: Iterative Changes

Instead of using a complex mathematical formula to rearrange all points at once, I use an iterative process. The dataset evolves through thousands of tiny, random modifications until the desired shape is reached.

The basic loop of the algorithm follows these four steps:

(1) **Random Selection:** A single random point is picked from the current dataset.
(2) **Movement:** the point is moved by a small, random distance in a random direction.
(3) **Evaluation:** The new statistics for the dataset are calculated (such as mean, standard deviation, and correlation) to see how they have changed. Additionally, the distance from the moved point to the nearest target shape point is compared against the distance before the move.
(4) **Decision:**
   - If the new statistics are still within a very small allowed margin of the original values *and* the point has moved closer to the target shape, the move is kept.
   - If the move causes the statistics to deviate too much from the original values, or moves the point further away from the target shape, it is discarded and the point returns to its previous position.

Although each individual movement is random in direction, the selective acceptance of moves that bring points closer to the target creates a directed drift over many iterations. By repeating this process many times, the points gradually shift toward the target shape while the overall statistical properties of the dataset remain virtually unchanged.

2.2 Avoiding Local Minima: Simulated Annealing

The simple "move and check" loop described above has one major weakness: the algorithm can get "stuck." This happens when any small move would temporarily make the statistics worse, even if that move is necessary to eventually reach the target shape. In optimization, this is called a *local minimum*.

To solve this, I implemented a **Simulated Annealing** strategy. The key idea is to occasionally accept moves that are "bad" (i.e., moves that slightly violate the statistical constraints or move away from the target shape).

The probability of accepting such a move is evaluated using the classic Metropolis acceptance criterion [4], which depends on a variable called **Temperature** (T):

- **At the beginning:** The temperature is high. The algorithm is "brave" and frequently accepts bad moves to explore the space and avoid getting stuck.
- **Over time:** The temperature slowly cools down. The algorithm becomes more "cautious" and eventually only accepts moves that strictly improve the result.

This cooling process ensures that the points can overcome obstacles on their way to the final target shape.

3 Implementation Details

In this section, I outline the technical realization of the algorithm. The focus of the implementation was not only on correctness but also on performance, as the iterative nature of the process requires millions of statistical calculations.

3.1 Language and Tools

The project is implemented in **C++23**. This choice was made for several reasons:

- **Performance:** C++ allows for low-level optimizations, which are crucial when running millions of iterations per second.
- **Modern Features:** The C++23 standard provides improved headers and features that simplify the management of data structures and random number generation.
- **Standard Library:** I primarily used the Standard Template Library (STL) to ensure the code remains portable and maintainable.

3.2 Data Structures

To represent the 2D datasets, I used a simple yet efficient approach. Each dataset is stored as a `std::vector` of a custom `Point` struct, containing two double-precision floating-point numbers (x and y). This ensures that the points are stored contiguously in memory, which optimizes cache performance during the frequent statistical updates.

3.3 Optimization: Incremental Statistical Updates

A naive implementation of the algorithm would recalculate the mean, variance, and correlation from scratch after every single move. For a dataset with n points, this would result in a time complexity of $O(n)$ per iteration. Given that the algorithm requires millions of iterations, this approach is inefficient.

To optimize the process, I implemented **incremental updates**. Instead of summing all points repeatedly, the algorithm maintains the current sums of x, x^2, y, y^2, and the product xy. When a point (x_{old}, y_{old}) is moved to (x_{new}, y_{new}), the sums are updated as follows:

$$\sum x_{new_total} = \sum x_{old_total} - x_{old} + x_{new}$$

This reduces the complexity of the statistical evaluation to $O(1)$ per iteration. This optimization significantly increases the number of iterations the algorithm can perform per second, allowing it to converge on complex shapes much faster.

3.4 Random Number Generation

Since the algorithm relies heavily on random shifts, the quality and speed of the random number generator are essential. I used the `std::mt19937` (Mersenne Twister) engine provided by the C++ `<random>` header. To generate the moves, a

`std::uniform_real_distribution` is used to ensure that every direction and distance within the defined step size has an equal probability of being chosen.

3.5 Spatial Indexing

Without optimization, computing the distance between a moved point and the target shape requires iterating over all m target points to find the nearest one, resulting in an $O(m)$ cost per iteration. Since this happens up to billions of times, this quickly becomes a bottleneck.

To avoid this, I precompute a **distance lookup table** during initialization. The coordinate space $[0, 100]^2$ is split into a 101×101 grid, where each cell stores the minimum Euclidean distance to the nearest target point. At runtime, a distance query is just a single array access:

$$d(p) = \texttt{grid}[\text{round}(p_x)][\text{round}(p_y)]$$

This brings the per-iteration distance cost from $O(m)$ down to $O(1)$, at the cost of about 80 KB of memory, which is small enough to fit in the CPU's L2 cache. Points that leave the $[0, 100]^2$ area get an extra penalty based on how far out they are, so they are pushed back inward.

3.6 Parallelization with OpenMP

I parallelized the optimization using OpenMP. Instead of trying to parallelize the inner loop of a single chain, which would require synchronization after every iteration, I run k independent chains in parallel, each with its own copy of the dataset and a unique random seed.

After all chains finish, the best result (lowest energy) is kept:

$$\text{result} = \arg \min_{c \in \{1,\ldots,k\}} E_c$$

Since the chains are fully independent, no communication or locking is needed during execution. In practice, the wall-clock time stays roughly the same regardless of thread count, while the result quality improves because more chains means more chances of finding a good solution.

3.7 Vectorization

The main loop processes one point per iteration: pick a random point, move it, check statistics, accept or reject. This sequential, branching pattern does not lend itself well to Single Instruction, Multiple Data (SIMD) vectorization.

The code is compiled with `-O3 -march=native`, so the compiler can still auto-vectorize simpler parts like the initial statistics computation and the grid precomputation. I chose not to use explicit SIMD intrinsics, since the main speedup here comes from the $O(1)$ algorithmic optimizations, not from instruction-level parallelism.

4 Evaluation Setup

To evaluate the performance and effectiveness of the algorithm, I conducted several experiments. The goal was to transform a standard dataset into various target shapes while monitoring the statistical deviation and the time required to reach the target shape. For all performance benchmarks, the following system was used:

- **OS:** macOS Tahoe 26.3.1
- **CPU:** Apple M4 Max
- **RAM:** 128 GB
- **Compiler:** GCC 15.2.0 (with `-O3` and `-fopenmp` flags)

4.1 Test Datasets and Target Shapes

I used the well-known *Datasaurus Dozen* as a starting point. The target shapes for the transformation included:

- A **Circle** to test uniform distribution along a curve.
- A **Star** to evaluate how the algorithm handles sharp angles and "empty" space.
- A **Horizontal Line** to see how the algorithm reacts when one dimension (y) must be highly constrained.
- A **Sierpinski Triangle** ($N = 1420$) to test a high-density fractal geometry that, to the best of my knowledge, has not been used as a target shape in prior work. This shape serves two purposes: it evaluates the algorithm's ability to handle complex, self-similar structures, and it tests scalability with a significantly larger point set (10× the standard Datasaurus size).

4.2 Parameters

For all tests, the following parameters were kept constant unless otherwise noted:

- **Number of points:** 142 (standard Datasaurus size), 1,420 for the high-density Sierpinski experiment.
- **Allowed error (ϵ):** 0.003 for mean, standard deviation, and correlation.
- **Iterations:** 10^9 per chain.

4.3 Results

The algorithm successfully transformed the initial Dinosaur dataset ($N = 142$) into clearly recognizable star, circle, and line geometries (see Figures 1, 2, and 3). Despite the massive visual changes, all descriptive statistics remained essentially frozen over 10^9 iterations with a strict threshold of $\epsilon = 0.003$.

Quantitative results, documented in Tables 1 through 3, demonstrate that mean deviations were kept under 5×10^{-3} and the Pearson correlation was preserved within 2.9×10^{-3} of the baseline. The line transformation (Fig. 3) is a particularly interesting case: despite the points forming a clear diagonal, the algorithm preserved the original near-zero correlation by carefully distributing them along the line.

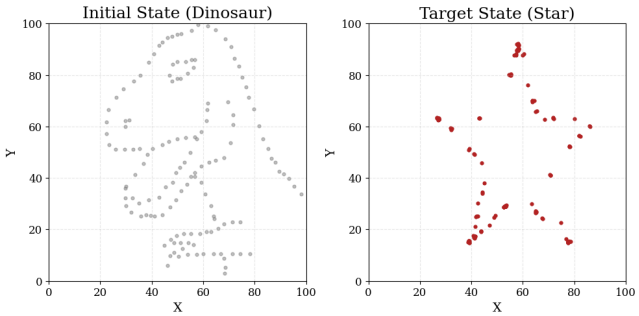

Figure 1: Visual transformation: Dinosaur to Star.

Metric	Dino	Star	Diff.
Mean X	54.2633	54.2624	$8.0e-04$
Mean Y	47.8323	47.8315	$2.1e-03$
Std X	16.7651	16.7657	$7.0e-04$
Std Y	26.9354	26.9354	$2.2e-03$
Corr	-0.0645	-0.0666	$2.2e-03$

Table 1: Statistical comparison: Dinosaur vs. Star.

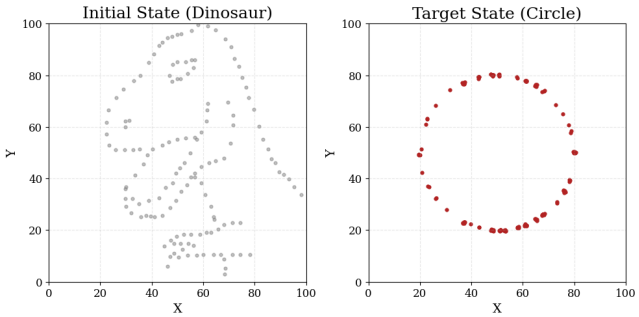

Figure 2: Visual transformation: Dinosaur to Circle.

Metric	Dino	Circle	Diff.
Mean X	54.2633	54.2605	$2.7e-03$
Mean Y	47.8323	47.8328	$3.0e-03$
Std X	16.7651	16.7629	$2.6e-03$
Std Y	26.9354	26.9324	$3.0e-03$
Corr	-0.0645	-0.0639	$2.9e-03$

Table 2: Statistical comparison: Dinosaur vs. Circle.

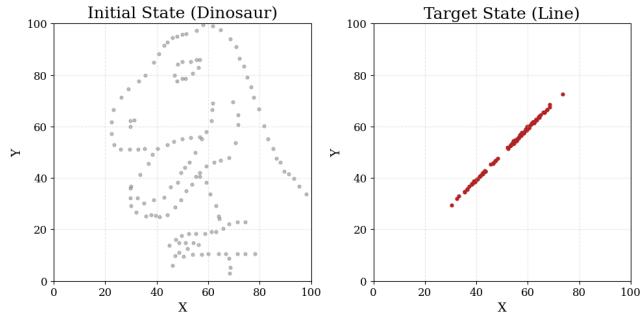

Figure 3: Visual transformation: Dinosaur to Line.

Metric	Dino	Line	Diff.
Mean X	54.2633	54.2603	$3.0e-03$
Mean Y	47.8323	47.8352	$3.0e-03$
Std X	16.7651	16.7631	$2.8e-03$
Std Y	26.9354	26.9332	$2.6e-03$
Corr	-0.0645	-0.0617	$2.7e-03$

Table 3: Statistical comparison: Dinosaur vs. Line.

4.4 Sierpinski Fractal and Scalability

To evaluate the scalability of the C++ implementation, a high-density experiment ($N = 1,420$) was conducted using a Sierpinski triangle fractal (Fig. 4). The higher density allowed the solver to form the fine details of the fractal, which would be invisible with fewer points.

As shown in Table 4, all statistics were preserved with the same precision as in the lower-density cases. This shows that the $O(1)$ update logic works just as well when the dataset is ten times larger.

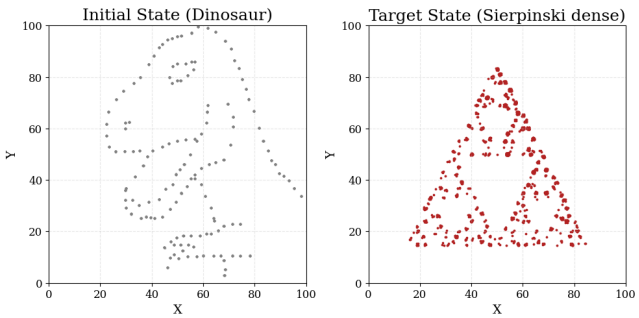

Figure 4: High-density transformation: Dinosaur to Sierpinski fractal ($N = 1420$).

Metric	Dino (Dense)	Sierpinski	Diff.
Mean X	54.2634	54.2616	$1.7e-03$
Mean Y	47.8323	47.8311	$1.2e-03$
Std X	16.7120	16.7119	$1.2e-03$
Std Y	26.8499	26.8493	$1.6e-03$
Corr	-0.0645	-0.0664	$1.9e-03$

Table 4: Statistical comparison for the high-density Sierpinski experiment.

4.5 Performance and Convergence

Due to the $O(1)$ incremental update optimization, the algorithm achieved high iteration speeds. Table 5 summarizes the performance metrics on the test system.

The throughput varies noticeably across target shapes: Star and Sierpinski dense achieve around 3.5×10^7 iterations per second, while Circle and Line are slower at roughly 2.5×10^7. This difference is likely caused by the acceptance rate of the optimizer. For geometrically simpler targets like Star, more proposed moves are accepted, leading to fewer wasted iterations and better pipeline utilization. For harder targets like Line, the algorithm rejects most moves, which increases branch misprediction overhead.

Despite this variation, the benchmark data confirms the theoretical $O(1)$ complexity of the incremental updates: the Sierpinski dense target uses ten times more points ($N = 1420$ vs. $N = 142$) yet runs in 27.5 s, which is comparable to Star (28.7 s) with the same acceptance characteristics. All configurations complete 10^9 iterations in under 45 seconds.

Table 5: Performance (16 chains, averaged over 5 runs).

Target Shape	Total Iterations	Time (ms)	Iterations/sec
Star	10^9	28,742	3.48×10^7
Circle	10^9	38,806	2.58×10^7
Line	10^9	41,385	2.42×10^7
Sierpinski dense	10^9	27,538	3.63×10^7

5 Conclusion

In this project, I successfully implemented an algorithm capable of generating 2D datasets with specific visual shapes while preserving their statistical properties. By using a Simulated Annealing approach, the algorithm can handle difficult transformations and avoid getting stuck where simpler methods would fail.

The implementation in C++23, combined with the incremental statistical updates and a precomputed distance grid, proved to be efficient. The reduction of complexity from $O(n)$ to $O(1)$ per iteration allowed the processing of up to billions of moves in under 45 seconds. The throughput ranged from 2.4×10^7 to 3.6×10^7 iterations per second depending on the target shape, with geometrically harder targets showing lower acceptance rates and correspondingly longer runtimes. The multi-chain OpenMP parallelization further improves solution quality by running several independent annealing chains at the same time. Future work could involve expanding this approach to higher-dimensional data or investigating adaptive cooling schedules.

Availability

The C++ source code, including the CMake build scripts and the benchmark datasets, is publicly available on GitHub at: https://github.com/Smurado/Algorithm-Engineering.

References

[1] Francis J Anscombe. Graphs in statistical analysis. *The american statistician*, 27(1):17–21, 1973.

[2] Justin Matejka and George Fitzmaurice. Same stats, different graphs: generating datasets with varied appearance and identical statistics through simulated annealing. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, pages 1290–1294. ACM, 2017.

[3] Scott Kirkpatrick, C Daniel Gelatt, and Mario P Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[4] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.